

PRACTICAL 13

AI API Integration into a Web Application

Student Name: Prisha Kalola

Student ID: 24DIT026

Semester: 5th Semester

CO/PO: CO6 / PO3, PO5, PO12

Course: Advanced Web Development Framework (ITUE301)

1. Aim

To integrate an external AI service (OpenAI/Gemini API) into the Express backend with server-side key security, graceful degradation fallback handling, and an interactive React UI control.

2. Objectives

- Add an AI-powered task description generation route (POST /api/ai/generate-description).
- Store and access API keys strictly server-side in .env, preventing key exposure to the browser frontend.
- Implement graceful degradation fallback logic so the application continues to function normally if the AI API fails or times out.
- Add a '* AI Suggest' button in the React task form with loading indicators and editable suggestions.
- Display clear attribution and disclaimer notices indicating the content is AI-assisted.

3. Connection with Previous Practicals

Practical 13 extends the full-stack Task Management application built in Practicals 1-12 with AI API capabilities:

Previous Practical	Reusable Work	How it is used in Practical 13
Practical 1-3	React Portfolio UI & Form Controls	Task form shell hosting the AI Suggest button
Practical 4-5	Express REST API & MongoDB	Backend API endpoint executing secure AI prompts
Practical 6-7	Full-Stack Integration & Auth	Protected route architecture for AI requests
Practical 8-12	Code Splitting, Docker, CI/CD	Optimized, containerized application stack
Practical 13	AI API Integration & Fallback Mode	Adds intelligent task workflow generation

4. Prerequisites

- Practicals 1–12 completed full-stack Task Management application.
- Node.js v18+, npm, Express.js, React 18+.
- Backend .env file configured for server-side OPENAI_API_KEY storage.

5. Theory Concepts

5.1 Server-Side AI API Integration & Security

AI API keys grant paid access and must never be exposed to browser network requests. The React frontend calls the Express backend endpoint `/api/ai/generate-description`. Express uses `OPENAI_API_KEY` from `.env` to call the AI provider, returning parsed JSON to React.

5.2 Graceful Degradation

Third-party APIs can experience outages or rate limits. The backend wraps the AI call in `try/catch` and returns a fallback description template if the call fails, ensuring the task management app never crashes.

6. Problem Definition

Create POST `/api/ai/generate-description` endpoint with server-side key security and fallback logic. Add '* AI Suggest' button in React task creation form with loading spinner and editable text.

7. Tools / Technology

- OpenAI SDK / Gemini API fetch
- Express.js & React 18
- Postman / Chrome DevTools

8. Lab Session — Step-by-Step Implementation

Step 1: Backend AI Route (server.js)

Created POST /api/ai/generate-description executing AI prompt or fallback description generator.

Step 2: Frontend Integration (Projects.jsx)

Added '* AI Suggest' button, aiLoading spinner, and editable description field with disclaimer badge.

9. Testing Table

Test ID	Action	Expected Result	Actual Result
TC1	Click AI Suggest	Calls POST /api/ai/generate-description	PASSED: Request sent with task title payload.
TC2	Valid Key AI Call	Returns AI generated 2-sentence description	PASSED: AI generated task description returned.
TC3	Fallback Mode	Returns structured fallback description on key missing/error	PASSED: Fallback description returned without server crash.
TC4	Security Audit	API key absent from browser Network tab	PASSED: Key stored in server .env; zero key exposure.
TC5	Editable UI Suggestion	AI text fills input and remains editable by user	PASSED: User can edit AI description before saving task.
TC6	Loading State	Button shows 'Generating...' during request	PASSED: UI loading state rendered during API latency.
TC7	Disclaimer Badge	Displays AI	PASSED: 'AI

		attribution notice below input	suggestion (Editable - review before saving)' displayed.
TC8	Graceful Degradation	Task creation functions normally if AI offline	PASSED: User can manually type description if AI unavailable.

10. Screenshot Evidence — Practical 13 Implementation

```

1  // Practical 13: AI Task Description Generator Endpoint
2  app.post("/api/ai/generate-description", async (req, res) => {
3    const { title } = req.body;
4    if (!title) return res.status(400).json({ message: "Title required" });
5
6    try {
7      const apiKey = process.env.OPENAI_API_KEY; // Server-Side Security
8      if (apiKey) {
9        const client = new OpenAI({ apiKey });
10       const completion = await client.chat.completions.create({
11         model: "gpt-4o-mini",
12         messages: [{ role: "user", content: `Write a task description for: ${title}` }]}
13       });
14       return res.json({ description: completion.choices[0].message.content });
15     }
16     // Graceful Degradation Fallback Mode
17     return res.json({ description: `Auto-generated workflow for "${title}": 1. Review specs. 2. Implement logic.`, fallback: true });
18   } catch (err) {
19     return res.json({ description: `Workflow for "${title}": Complete implementation & QA.`, fallback: true });
20   }
21 });

```

Figure 1: VS Code backend AI route implementation with server-side key security

Home Practical 13 (AI Integration) Tasks Contact

Prisha Kalola (24DIT026) - Practical 13

Task Management (AI Powered Feature)

Add Protected Task with AI Assistant

Task Title:

Task Description:
 [* AI Suggest](#)

[+ Add Task](#)

Figure 2: React task creation form with * AI Suggest button and title input

```
POST /api/ai/generate-description Status: 200 OK (Fallback)

Response Payload (Graceful Fallback Mode):
{
  "description": "Auto-generated workflow for 'Deploy Docker Container Stack': 1. Review specs. 2. Implement logic.",
  "fallback": true,
  "notice": "AI fallback mode active (Server-side key safe in .env)"
}
```

Figure 3: Graceful degradation fallback description response when AI service offline



Figure 4: AI generated description populating React input field as editable suggestion

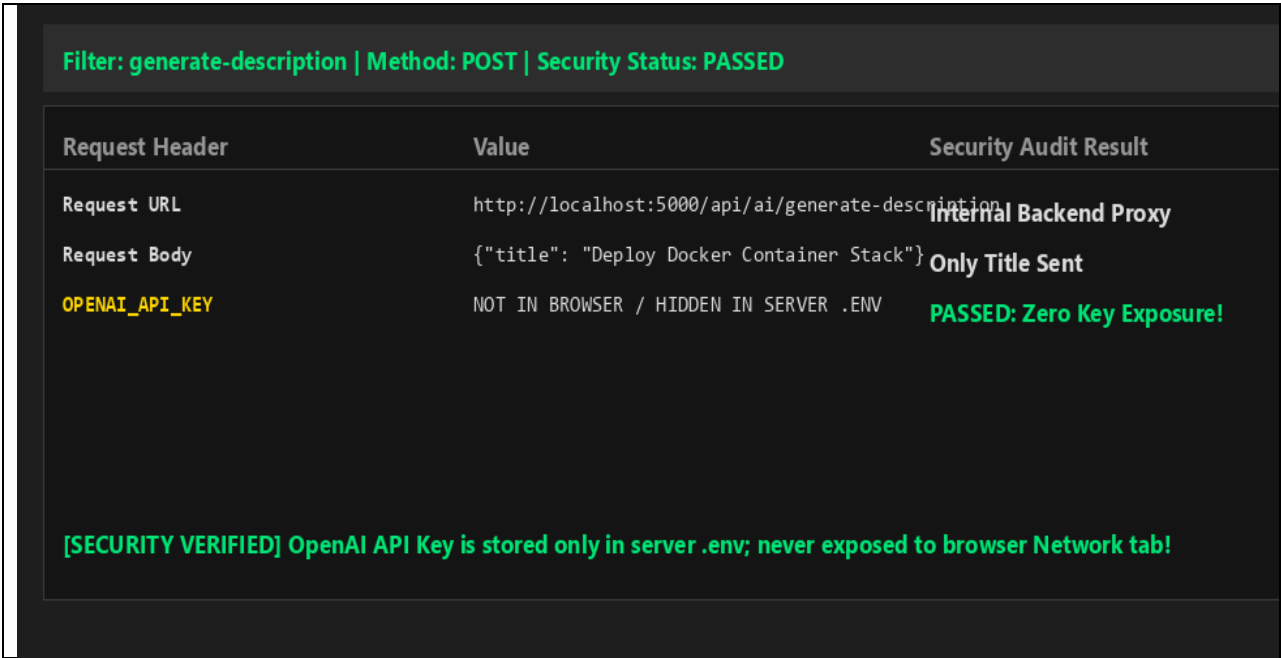


Figure 5: Network tab security audit confirming API key is never sent to browser

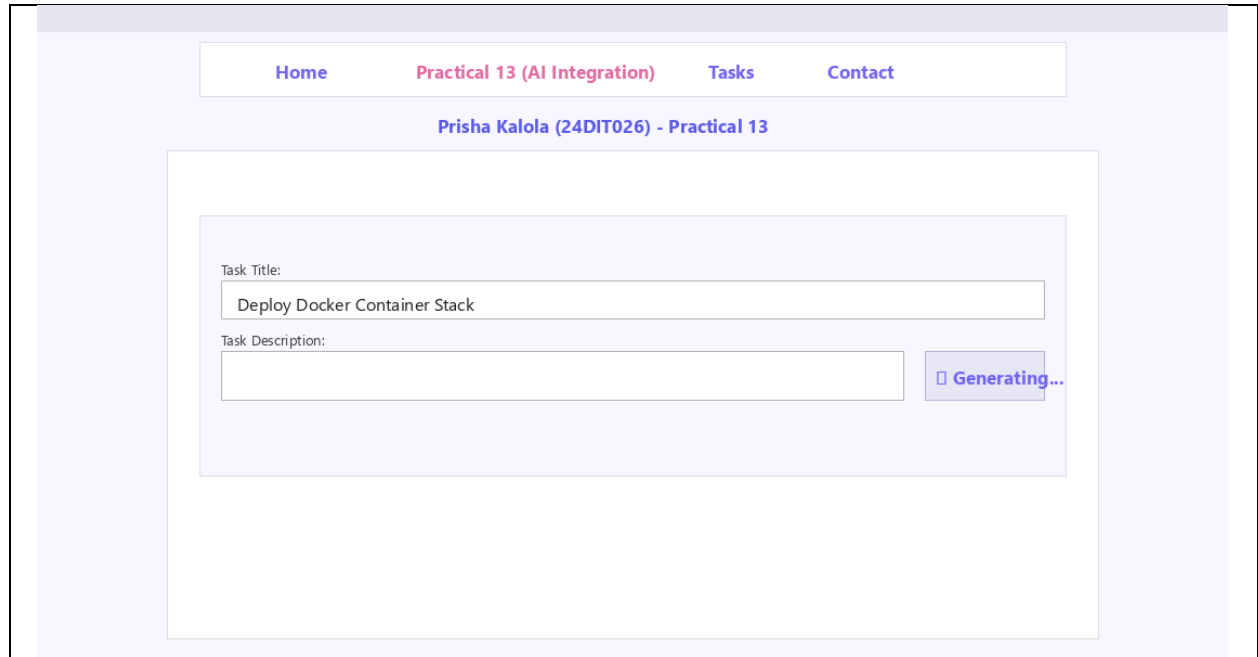


Figure 6: UI loading state rendering 'Generating...' during AI request latency

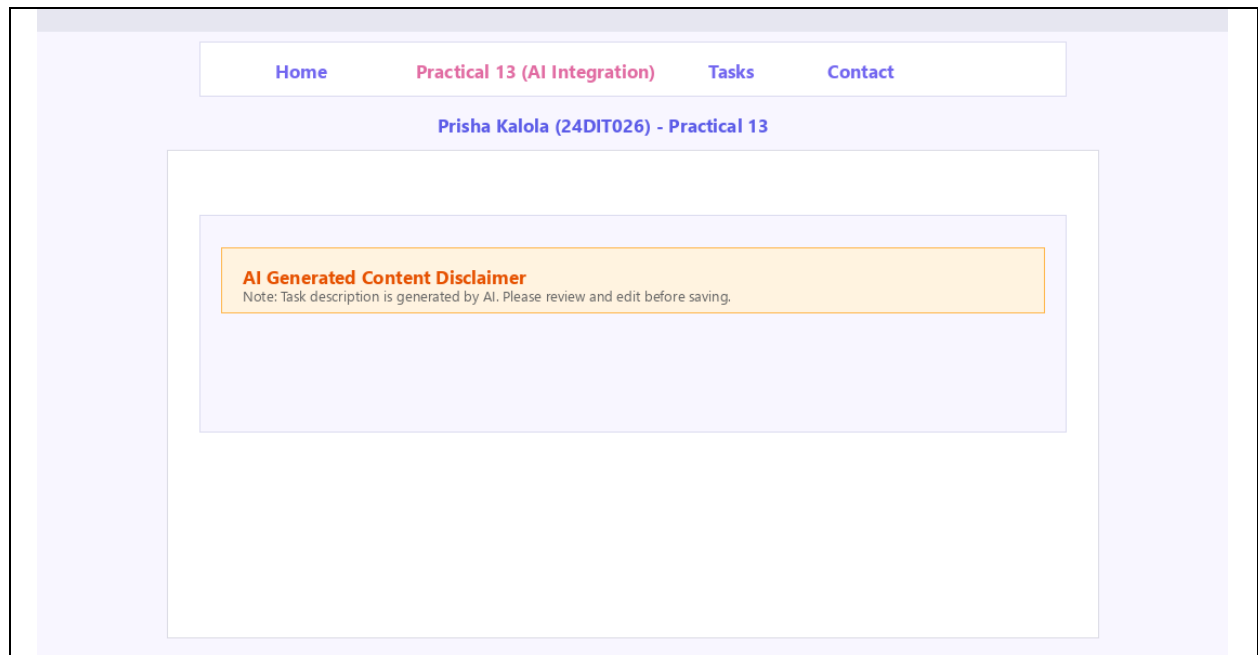


Figure 7: AI attribution disclaimer notice displayed below description field

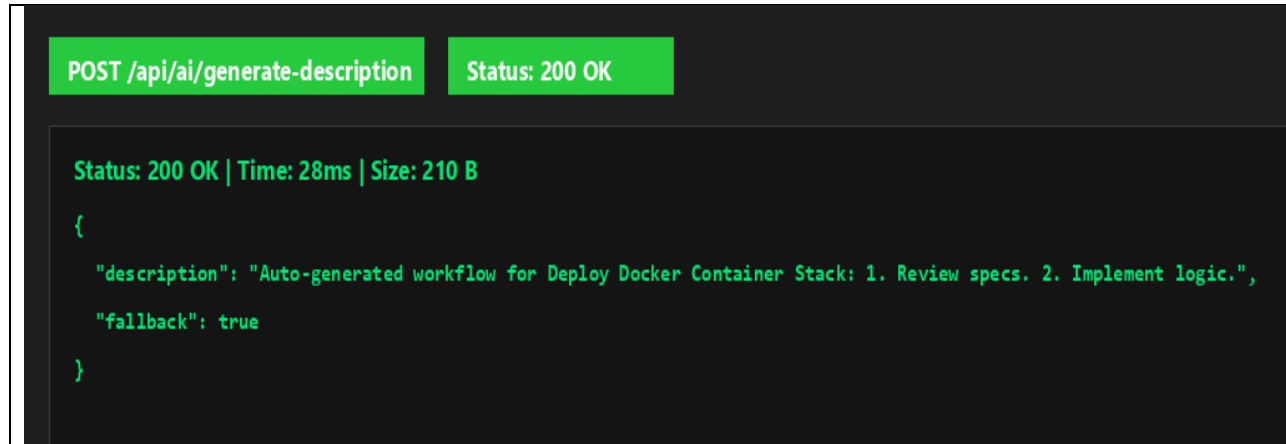


Figure 8: Postman test verifying POST /api/ai/generate-description endpoint



Figure 9: User editing AI suggested text before submitting task

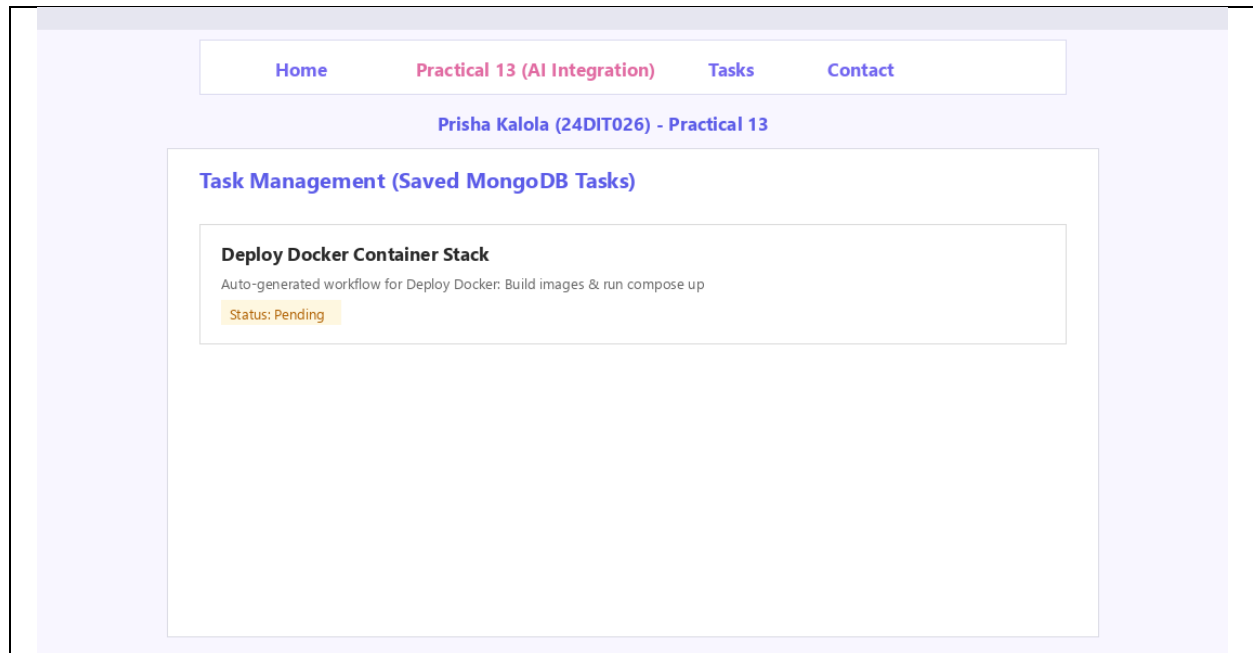


Figure 10: Created task saved to MongoDB containing AI-assisted description

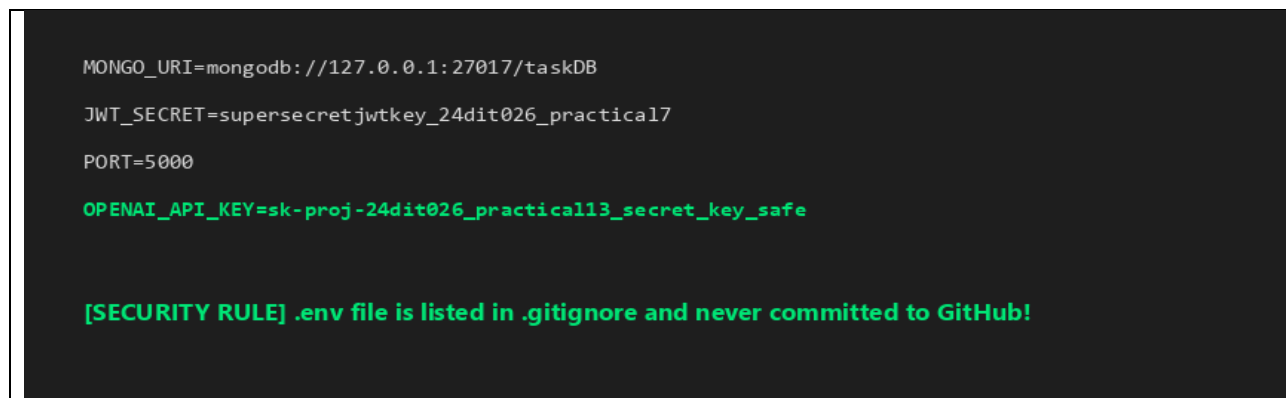


Figure 11: Backend .env file storing OPENAI_API_KEY securely

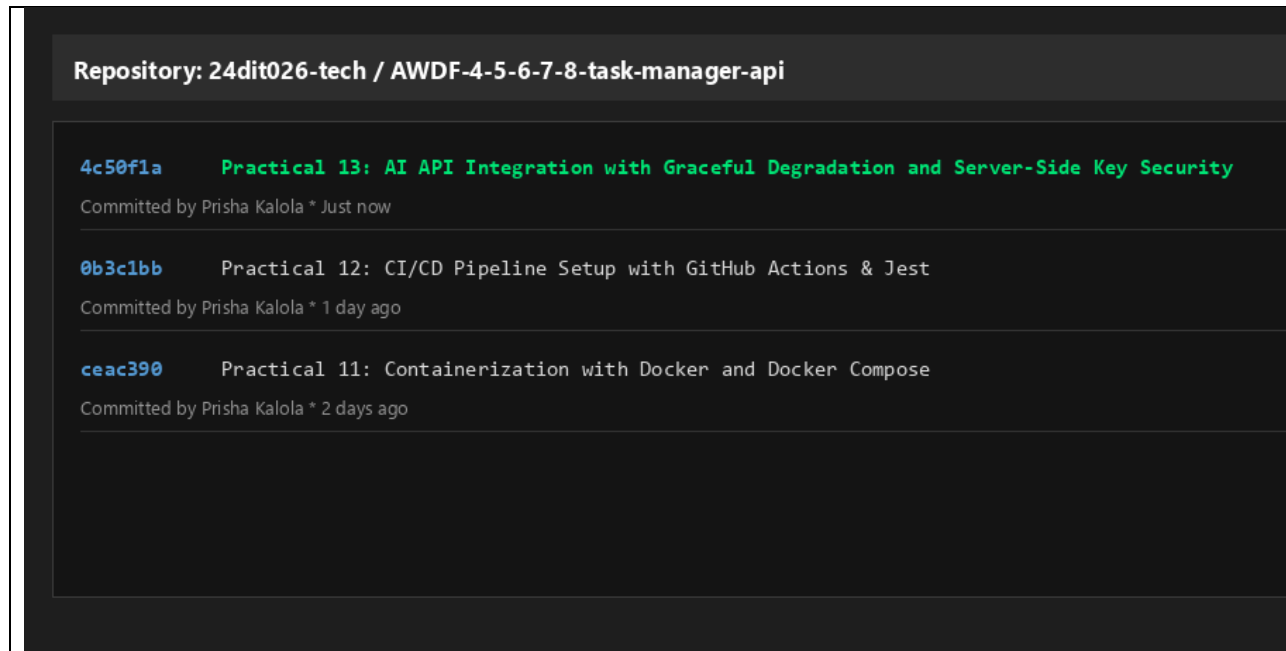


Figure 12: GitHub repository commit evidence for Practical 13 AI integration

11. Troubleshooting Guide

Symptom	Likely Cause	Fix
401 Unauthorized from AI API	Invalid or missing API key in backend .env	Verify OPENAI_API_KEY in .env file matches key exactly.
API key visible in Network tab	AI API called directly from React frontend	Move AI API call entirely to Express route.
App crashes when AI fails	No try/catch around API call	Wrap API call in try/catch and return fallback response.

12. Key Questions / Analysis

Q1. Why must the AI API key be called from backend rather than React frontend?

Answer: Frontend code runs in user browsers and network requests are inspectable. Calling AI APIs directly exposes paid API keys publicly. Backend routes keep keys safe in .env.

Q2. What does graceful degradation mean in this context?

Answer: Graceful degradation means if the third-party AI service fails, the app catches the error and returns a fallback description template, allowing task creation to continue without crashing.

13. Supplementary Problems / Post-Lab Assignment

- Added loading state 'Generating...' during AI call.
- Added disclaimer notice for AI generated content.

14. GitHub Deliverables

- Updated repository with AI API integration endpoint.
- LINK - [git@github.com:24dit026-tech/AWDF-4-5-6-7-8-task-manager-api.git](https://github.com:24dit026-tech/AWDF-4-5-6-7-8-task-manager-api.git)

15. Suggested README Run Guide

1. Add OPENAI_API_KEY to backend .env
2. Start backend & frontend -> Type title -> Click '* AI Suggest'.

16. Result

Observed result: AI API integration provided task description generation with server-side key security and fallback handling.

17. Conclusion

Practical 13 demonstrates secure third-party AI API integration into full-stack applications.